

PROGRAMLAMA DİLLERİ

ÇALIŞMA SORULARI İDEAL CEVAPLAR

Soru 1

Programlama dili, makine dili, yorumlayıcı ve derleyici kavramlarını açıklayınız.

İdeal Cevap:

Programlama dili, bir problemin çözümünü bilgisayarda ifade etmek için kullanılan kurallı bir dildir. Bu dilde hem insanların anlayabileceği semboller ve ifadeler bulunur hem de bu ifadelerin bilgisayar tarafından çalıştırılabilmesi için belirli sözdizim ve anlam kuralları vardır. Yani programlama dili, insanın problem çözme düşüncesi ile bilgisayarın işlem yapma biçimi arasında köprü kurar.

Makine dili ise bilgisayarın doğrudan anlayabildiği en düşük düzeyli gösterimdir. Komutlar ikili sistemde veya işlemcinin anlayacağı biçimde temsil edilir. Yüksek düzeyli bir dilde yazılan programın çalışabilmesi için en sonunda makine diline çevrilmesi gerekir.

Derleyici, yüksek düzeyli kaynak kodun tamamını ya da büyük bir bölümünü çalıştırmadan önce makine koduna çeviren programdır. Derleme işleminden sonra program genellikle daha hızlı çalışır; çünkü çalışma sırasında her satır yeniden çevrilmez. C ve C++ gibi dillerde derleme yaklaşımı belirgindir.

Yorumlayıcı ise programı çalıştırırken deyimleri sırayla okuyup yorumlayan sistemdir. Programın tamamını önceden makine koduna çevirmek yerine, çalışma sırasında satır satır veya deyim deyim işler. Bu yaklaşım geliştirme ve deneme açısından esneklik sağlar; ancak çoğu durumda derlenmiş programa göre daha yavaş çalışabilir. Python gibi diller bu yaklaşımla ilişkilendirilebilir.

Kısaca programlama dili çözümü ifade eder, makine dili bilgisayarın doğrudan anladığı biçimdir, derleyici kaynak programı çalıştırmadan önce çevirir, yorumlayıcı ise programı çalışırken yorumlayarak yürütür.

Konu PDF ve Sayfa Bilgisi: PD0.pdf içinde programlama dili ve makine dili kavramları özellikle s.13-17 ve s.21'de, dil çevrimi s.25 civarında, yorumlayıcılar s.27'de anlatılır. PD1.pdf içinde implementasyon metodları, derleme ve yorumlama yaklaşımı s.110-116 arasında geçmektedir.

Soru 2

Değişkenlerde değer, tip, yaşam süresi ve kapsam kavramlarını açıklayınız.

İdeal Cevap:

Değişken, programlama dilinde bir veya daha fazla bellek hücresinin soyutlanmış halidir. Programcı değişkene bir isim verir ve bu isim üzerinden bellekteki değere ulaşır. Değişkenin adresi l-value, o adreste tutulan içerik ise r-value olarak düşünülebilir.

Değer, değişkenin bellekte tuttuğu bilgidir. Örneğin `int sayi = 5;` ifadesinde `sayi` değişkeninin değeri 5'tir. Program çalışırken değişkenin değeri atama işlemleriyle değiştirilebilir.

Tip, değişkenin hangi değerleri alabileceğini ve bu değerler üzerinde hangi işlemlerin yapılabileceğini belirler. Örneğin `int` tipindeki bir değişken tamsayı değerler alır ve sayısal işlemlerde kullanılabilir. Tip bilgisi, programdaki hataların daha erken fark edilmesine yardım eder.

Yaşam süresi, bir değişkenin bellekle ilişkili kaldığı süredir. Örneğin bir fonksiyon içinde tanımlanan yerel değişken, fonksiyon çalışmaya başladığında oluşur ve fonksiyon bitince genellikle yok olur. Global değişkenler ise program boyunca bellekte kalabilir.

Kapsam, değişken isminin programın hangi bölümlerinde geçerli olduğunu gösterir. Bir değişken yalnızca tanımlandığı blok içinde geçerliyse yerel kapsamlıdır. Programın birçok yerinden erişilebiliyorsa global kapsamlıdır. Kapsam kavramı, aynı isimli değişkenlerin karışmasını önler ve programın anlaşılabilirliğini artırır.

Konu PDF ve Sayfa Bilgisi: Bu konu PD5.pdf Bölüm 5 içinde geçmektedir. Değişkenin bellek hücresinin soyutlaması olması s.11’de, isim/adres/değer/tip/yaşam süresi/kapsam kavramları s.12-15 arasında anlatılır. Veri tipi kavramına giriş ise s.23-24 civarındadır.

Soru 3

Değişkenler, veri tipleri, bağlama (binding), kapsam (scope) ve tip kontrolü kavramlarını açıklayınız.

İdeal Cevap:

Değişken, program içinde veri saklamak için kullanılan isimlendirilmiş bellek alanı olarak düşünülebilir. Değişkenin adı, değeri, tipi, adresi, yaşam süresi ve kapsamı vardır. Bu özellikler programın nasıl çalışacağını doğrudan etkiler.

Veri tipi, bir değerler kümesini ve bu değerler üzerinde yapılabilecek işlemleri tanımlar. Örneğin tamsayı tipi ile toplama ve çıkarma yapılabilir; karakter dizisi tipi ise metinsel verileri tutar. Veri tipleri programın ifade gücünü ve güvenilirliğini artırır.

Bağlama, bir program elemanı ile bir özellik arasında ilişki kurulmasıdır. Örneğin bir değişken isminin bir veri tipiyle, bir değer bir değişkenle veya bir operatörün belirli bir anlamla ilişkilendirilmesi bağlamadır. Bağlama çalışma zamanından önce yapılırsa statik, çalışma sırasında yapılırsa dinamik bağlama olarak değerlendirilir.

Kapsam, bir ismin programın hangi bölümünde geçerli olduğunu belirtir. Statik kapsamda hangi değişkenin kullanılacağı program metnine bakılarak belirlenir. Dinamik kapsamda ise değişken arama, çalışma zamanındaki çağrı zincirine göre yapılır.

Tip kontrolü, programda kullanılan işlemlerin veri tiplerine uygun olup olmadığının denetlenmesidir. Örneğin sayısal işlem bekleyen bir yerde metin kullanılırsa tip hatası oluşabilir. Tip kontrolü derleme zamanında yapılırsa hatalar daha erken yakalanır; çalışma zamanında yapılırsa daha esnek ama daha riskli bir yapı oluşabilir.

Konu PDF ve Sayfa Bilgisi: Bu cevap PD5.pdf ve PD6.pdf’e dayanmaktadır. Değişken ve veri tipi kavramları PD5.pdf s.11-15 ve s.23-24’te; bağlama kavramı s.75-80’de; tip bağlama s.83-85 civarında; kapsam ve statik/dinamik kapsam kuralları s.113-123 arasında anlatılır. Tip güvenliği ve örtülü tip dönüşümleri PD6.pdf s.137-145 arasında geçmektedir.

Soru 4

Sonuç ile çağırma yöntemine göre aşağıdaki programı açıklayarak program çıktısını yazınız.

İdeal Cevap:

```
void plus(by-value int a, by-value int b, byresult int c)
{
    c = a + b;
}

void f()
{
    int x = 3; int y = 4; int z;
```

```
plus(x, y, z);  
write z;  
}
```

Bu soruda kullanılan yöntem sonuç ile çağırma. Sonuç ile çağırma altprogram çağırılırken gerçek parametreden resmi parametreye başlangıçta değer aktarılmaz. Bunun yerine, altprogram çalışmasını bitirdikten sonra resmi parametrenin son değeri gerçek parametreye kopyalanır.

Programda plus fonksiyonunda a ve b parametreleri by-value olarak verilmiştir. Bu yüzden x ve y değişkenlerinin değerleri a ve b'ye kopyalanır. $x = 3$ olduğu için $a = 3$, $y = 4$ olduğu için $b = 4$ olur. c parametresi ise byresult olarak tanımlanmıştır. Bu nedenle c'nin değeri fonksiyon sonunda gerçek parametre olan z'ye aktarılır.

Fonksiyon içinde $c = a + b$ işlemi yapılır. Burada $a + b = 3 + 4 = 7$ olur. Fonksiyon bittiğinde c'nin değeri z'ye kopyalanır. Bu yüzden write z; satırı 7 değerini yazar.

Program çıktısı: 7

Konu PDF ve Sayfa Bilgisi: Parametre aktarım yöntemleri PD8.pdf Bölüm 8'de anlatılır. Değer ile çağırma s.37-41 arasında, sonuç ile çağırma s.42-44 arasında, değer-sonuç ile çağırma s.47-49 arasında açıklanır. Altprogram çağırısı ve parametre kavramları PD8.pdf s.5-8 ve s.29 civarında da desteklenmektedir.

Soru 5

Nesneye yönelimli, fonksiyonel ve mantıksal programlama yaklaşımlarının temel özelliklerini yazınız ve karşılaştırınız.

İdeal Cevap:

Nesneye yönelimli programlama, programı sınıf ve nesneler üzerinden kurar. Veri ile o veri üzerinde çalışan metotlar aynı yapı içinde tutulur. Bu yaklaşımda kapsülleme, kalıtım ve çok biçimlilik gibi kavramlar önemlidir. Java, C++ ve C# bu yaklaşıma örnek verilebilir. Büyük sistemlerde modülerlik, tekrar kullanılabilirlik ve veri soyutlama açısından güçlüdür.

Fonksiyonel programlama, matematiksel fonksiyon düşüncesine dayanır. Aynı giriş verildiğinde aynı çıktının elde edilmesi beklenir. Değişken kullanımı ve atama işlemleri az olduğu için yan etkiler azaltılır. Lisp, Scheme, ML ve Haskell fonksiyonel dillere örnektir. Bu yaklaşımda problem, fonksiyonların birleştirilmesiyle çözülür.

Mantıksal programlama ise programı komutlar dizisi şeklinde değil, gerçekler ve kurallar şeklinde ifade eder. Programcı sonucun nasıl hesaplanacağını ayrıntılı yazmak yerine, ilişkileri ve kuralları tanımlar. Prolog bu yaklaşımın en bilinen örneğidir. Sistem, sorgulara cevap üretmek için mantıksal çıkarım yapar.

Karşılaştırırsak; nesneye yönelimli yaklaşım veriyi ve davranışı birlikte düzenler, fonksiyonel yaklaşım yan etkileri azaltarak fonksiyonları merkeze alır, mantıksal yaklaşım ise gerçekler ve kurallar üzerinden çıkarım yapar. Bu yüzden her yaklaşım farklı problem türlerinde daha avantajlı olabilir.

Konu PDF ve Sayfa Bilgisi: Nesneye yönelik programlama PD11.pdf Bölüm 11'de, özellikle s.3-16 arasında anlatılır. Fonksiyonel programlama PD14.pdf Bölüm 14'te, özellikle s.3-6 ve s.48-50 arasında geçer. Mantıksal programlama ve Prolog PD15.pdf Bölüm 15'te, özellikle s.3, s.20-28 ve s.39'da anlatılır.

Soru 6

BNF ile verilen gramerin EBNF ile tanımlanmış halini yazınız.

İdeal Cevap:

BNF’de aynı yapının iki farklı üretimle yazıldığını görüyoruz. İlk üretimde yalnızca if kısmı vardır. İkinci üretimde ise if kısmına ek olarak else kısmı da vardır. Yani else bölümü isteğe bağlıdır.

Bu nedenle EBNF’de ortak kısım bir kez yazılır, isteğe bağlı olan else <statement> kısmı köşeli parantez içinde gösterilir. EBNF gösterimi şu şekilde yazılabilir:

```
<eger_ifadesi> -> if(<expression>) <statement> [ else <statement> ]
```

Burada köşeli parantez içindeki bölüm isteğe bağlıdır. Yani ifade yalnızca if yapısı olarak da yazılabilir, if-else yapısı olarak da yazılabilir. EBNF’nin amacı tekrar eden veya isteğe bağlı kısımları daha kısa ve okunabilir biçimde göstermektir.

Konu PDF ve Sayfa Bilgisi: *PD3.pdf içinde BNF ve EBNF gösterimleri Bölüm 3 kapsamında anlatılır. Özellikle BNF/EBNF ile sözdizim tanımlama konuları s.30-38 ve devamındaki gramer gösterimi örnekleriyle ilişkilidir.*

Soru 7

Aşağıda verilen kod static ve dinamik kapsamda hangi çıktıları verir? Ayrı ayrı yazınız.

İdeal Cevap:

```
#include <stdio.h>
int a = 2;

void x(){
    int a = 4;
    int b = 9;
}

void y(){
    int a = 8;
    int c = 14;
}

int main()
{
    int a = 6;
    x();
    printf("%d", a);
    y();
    printf("%d", a);
    return 0;
}
```

Kodda global düzeyde int a = 2; tanımlıdır. x fonksiyonu içinde yerel olarak int a = 4; ve int b = 9; tanımlanmıştır. y fonksiyonu içinde de yerel int a = 8; ve int c = 14; vardır. main fonksiyonu içinde ise ayrıca yerel int a = 6; tanımlanmıştır.

Programda önce x() çağrılır. Ancak x fonksiyonu içindeki a yalnızca x fonksiyonunun yerel değişkenidir. main içindeki a değişkenini değiştirmez. Bu yüzden x() çağrısından sonra printf("%d", a); satırı main içindeki a değerini, yani 6’yı yazar.

Daha sonra y() çağrılır. y fonksiyonu içindeki a da sadece y fonksiyonuna aittir ve main içindeki a’yı değiştirmez. Bu nedenle ikinci printf de yine main içindeki a değerini yazar. Yani ikinci çıktı da 6’dır.

Statik kapsamda çıktı: 66

Dinamik kapsamda çıktı: 66

Burada statik ve dinamik kapsamın sonucu değiştirmemesinin nedeni, x ve y fonksiyonlarında main içindeki a'ya yönelik bir atama veya yazdırma işlemi yapılmamasıdır. Kapsam farkı genellikle bir fonksiyonun kendi içinde tanımlı olmayan bir değişkene erişmeye çalıştığı durumlarda belirginleşir.

Konu PDF ve Sayfa Bilgisi: Statik ve dinamik kapsam kavramları PD5.pdf Bölüm 5'te anlatılır. Kapsamın tanımı s.113'te, durağan kapsam bağlama s.114-123 arasında, dinamik kapsam bağlama ise PD5.pdf'in kapsam konusunun devamında anlatılmaktadır. Bu soru özellikle yerel/global değişken ve isim gizleme mantığıyla ilgilidir.

Soru 8

Aşağıdaki kod parçasında oluşabilecek tip problemlerini tespit ediniz, açıklayınız ve kodu düzeltiniz.

İdeal Cevap:

```
fiyat = "49.90"
adet = 3
toplam = fiyat * adet
kdv = toplam * 0.18
print("Ödenecek tutar: " + kdv)
```

Kodda fiyat değişkeni "49.90" şeklinde tırnak içinde yazıldığı için sayısal değer değil, string yani karakter dizisidir. adet ise tamsayıdır. toplam = fiyat * adet satırında Python'da string ile int çarpımı teknik olarak hata vermeyebilir; fakat bu işlem sayısal çarpma değil, string tekrar etme anlamına gelir. Yani burada mantıksal bir tip problemi vardır.

Asıl çalışma zamanı hatası kdv = toplam * 0.18 satırında oluşur. Çünkü toplam artık sayısal bir değer değil, string biçiminde tekrar edilmiş bir metindir. Bir string değer float bir değerle çarpılamaz. Bu durum tür uyumsuzluktur, otomatik tür dönüşümü değildir.

print("Ödenecek tutar: " + kdv) satırı da sorundur. Eğer kdv sayısal bir değer olsaydı bile string ile doğrudan birleştirilemezdi. Yazdırırken kdv değeri str(kdv) ile metne çevrilmeli ya da f-string kullanılmalıdır.

Bu hatalar Python'un otomatik olarak string değeri sayıya çevirmemesinden kaynaklanır. Yani burada güvenli bir type coercion yoktur; programcı açık tür dönüşümü yapmalıdır.

Düzeltilmiş kod şöyle yazılabilir:

```
fiyat = float("49.90")
adet = 3
toplam = fiyat * adet
kdv = toplam * 0.18
odenecek_tutar = toplam + kdv
print(f"Ödenecek tutar: {odenecek_tutar:.2f}")
```

Konu PDF ve Sayfa Bilgisi: Tip kontrolü, güçlü tiplendirme, tip uyumsuzluğu ve örtülü tip dönüşümü konuları PD6.pdf içinde anlatılır. Özellikle kuvvetli tiplendirme s.137 civarında, tip denetimi ve tip dönüşümleri s.139-145 arasında geçmektedir. Değişkenin tipi ve değeri kavramları PD5.pdf s.14-15 ile de ilişkilidir.

Soru 9

Akıllı trafik ışığı kontrol sistemi sorusundaki olay tabanlı yapı, token-lexeme analizi, istisna yönetimi ve bu kavramların ilişkilendirilmesini açıklayınız.

İdeal Cevap:

a) Olay tabanlı yapıda önce fonksiyonlar tanımlanır. arac_gecti(veri) fonksiyonu araç geçişi olayında çalışır ve gelen veri içinden plaka bilgisini alıp ekrana yazar. ariza() fonksiyonu ise sensör arızası olayında çalışır. olaylar adlı sözlükte olay adları ile bu olaylarda çalışacak fonksiyonlar eşleştirilmiştir. Program çalışınca önce

Sistem başlatıldı. yazılır. Sonra olaylar["aracGecti"] çağrılır ve plaka bilgisiyle birlikte Araç algılandı: 34ABC123 çıktısı oluşur. Son olarak olaylar["ariza"] çağrılır ve Sensör arızası! yazılır.

b) $yogunluk = 85 + hiz * 2$ ifadesinde sözcüksel analiz, ifadeyi lexeme denilen parçalara ayırır ve her parçayı token sınıfıyla eşleştirir. Bu analiz sonucunda $yogunluk$ bir tanımlayıcı, $=$ atama operatörü, 85 tamsayı sabiti, $+$ toplama operatörü, hiz tanımlayıcı, $*$ çarpma operatörü, 2 ise tamsayı sabiti olarak değerlendirilir.

Lexeme	Token
yogunluk	IDENT / Tanımlayıcı
=	ASSIGN OP / Atama operatörü
85	INT LITERAL / Tamsayı sabiti
+	ADD_OP / Toplama operatörü
hiz	IDENT / Tanımlayıcı
*	MULT_OP / Çarpma operatörü
2	INT LITERAL / Tamsayı sabiti

c) `veri_isle("yüksek")` çağrıldığında `int("yüksek")` işlemi yapılamaz. Çünkü "yüksek" sayıya çevrilebilen bir metin değildir. Bu nedenle `ValueError` oluşur. Kodda bu hata yakalanmadığı için program normal şekilde devam edemez. Bu durumda `try-except-finally` yapısı kullanılmalıdır. `except` bölümünde geçersiz veri yakalanır, `finally` bölümünde ise veri işleme denemesinin tamamlandığı bilgisi yazılabilir veya kullanılan kaynaklar kapatılabilir.

```
def veri_isle(sensor_verisi):
    try:
        yogunluk = int(sensor_verisi)
        return yogunluk * 2
    except ValueError:
        print("Geçersiz sensör verisi!")
        return None
    finally:
        print("Sensör verisi işleme denemesi tamamlandı.")
```

d) Trafik sistemi bağlamında olay tabanlı programlama, sensörden araç geçişi veya arıza gibi olaylar geldiğinde ilgili fonksiyonların çalışmasını sağlar. Program ayrıştırma, gelen veri veya komutun anlamlı parçalara ayrılıp sistem tarafından anlaşılmasını sağlar. İstisna yönetimi ise sensörden bozuk, eksik veya geçersiz veri geldiğinde sistemin tamamen çökmesini engeller. Bu üç yapı birlikte kullanıldığında sistem daha düzenli, güvenilir ve sürdürülebilir olur.

Konu PDF ve Sayfa Bilgisi: Bu soru üç farklı ders konusunu birleştirir. Token-lexeme ve ayrıştırma konusu PD4.pdf Bölüm 4'te, özellikle s.7-8 ve sözcüksel analiz örneklerinin geçtiği sayfalarda anlatılır. İstisna yönetimi PD13.pdf Bölüm 13'te, özellikle s.3-10 ve Java try-catch örnekleriyle ilişkilidir. Program ayrıştırma ve modüler düşünme PD10.pdf Bölüm 10'da, özellikle s.3-10 arasında yer almaktadır.

Soru 10

Kütüphane yönetim sistemi sorusundaki kontrol yapıları, altprogram, modüler yapı ve refactor işlemini açıklayınız.

İdeal Cevap:

a) İlk kodda `for` döngüsü kitaplar listesindeki her kitabı sırayla dolaşır. Bu, yineleme yapısıdır. `if kitap["musait"]` koşulu ise kitabın ödünç alınıp alınamayacağını kontrol eder. Eğer `musait` değeri `True` ise kitap ödünç alınabilir mesajı yazılır; değilse `else` bloğu çalışır ve kitabın mevcut olmadığı belirtilir. Bu yapı, yapısal programlamadaki yineleme ve seçim yapılarına örnektir.

b) `ceza_hesapla` fonksiyonu bir altprogramdır. `iade_tarihi`, `bugun` ve `gunluk_ceza` parametrelerini alır. `gunluk_ceza=2` ifadesi varsayılan parametredir; yani fonksiyon çağrılırken günlük ceza ayrıca verilmezse 2 TL

kabul edilir. Fonksiyon önce geciken gün sayısını hesaplar. Gecikme varsa geciken gün ile günlük cezayı çarpar ve sonucu döndürür. Gecikme yoksa 0 döndürür.

c) Verilen modüler yapıda uye_islemleri.py üye ekleme ve sorgulama işlemlerinden, kitap_islemleri.py kitap ekleme ve ödünç alma işlemlerinden, ceza_islemleri.py ise ceza hesaplama ve tahsil etme işlemlerinden sorumludur. Bu yapı modüler programlamaya uygundur çünkü her dosya belirli bir görevi üstlenmiştir. Böylece kod daha okunabilir olur, bakım kolaylaşır ve her modül ayrı ayrı test edilebilir.

d) kutuphane_yonet fonksiyonu modüler programlama açısından zayıftır; çünkü üye ekleme, kitap ödünç alma ve ceza hesaplama gibi farklı işleri tek fonksiyonda toplamıştır. Ayrıca bazı parametreler her işlemde kullanılmamaktadır. Bu durum fonksiyonun anlaşılmasını ve genişletilmesini zorlaştırır. Daha doğru yaklaşım, her işi ayrı fonksiyona bölmektir. Örneğin üye ekleme, kitap ödünç alma ve ceza yazdırma ayrı altprogramlar olmalıdır.

Refactor edilmiş örnek kod:

```
def uye_ekle(ad):
    print(ad + " sisteme eklendi.")

def kitap_odunc_al(ad, kitap):
    print(ad + " kitabı ödünç aldı: " + kitap)

def ceza_hesapla(gun, gunluk_ceza=2):
    return gun * gunluk_ceza

def ceza_yazdir(ad, gun):
    ceza = ceza_hesapla(gun)
    print(ad + " cezası: " + str(ceza) + " TL")

def kutuphane_yonet(islem, ad, kitap=None, gun=0):
    if islem == "ekle":
        uye_ekle(ad)
    elif islem == "odunc":
        kitap_odunc_al(ad, kitap)
    elif islem == "ceza":
        ceza_yazdir(ad, gun)
    else:
        print("Geçersiz işlem")
```

Konu PDF ve Sayfa Bilgisi: Kontrol yapıları ve yapısal programlama PD7.pdf Bölüm 7'de, özellikle s.3-14 ve seçim/yineleme yapılarının anlatıldığı kısımlarda geçmektedir. Altprogramlar ve parametreler PD8.pdf Bölüm 8'de, özellikle s.3-8 ve fonksiyon çağırımıyla ilgili s.29 civarında anlatılır. Modülerlik ve program ayrıştırma PD10.pdf Bölüm 10'da, özellikle s.3-10 ve veri soyutlama/modüler programlama konuları içinde yer alır.